Basics Day 3 — Session 3 (Hours 9–12)

Python Programming (Basic) • Comparisons, F-Strings, Text Processing & Debugging

Session 3 Overview

- Hour 9: Comparisons + boolean logic
- Hour 10: String formatting with f-strings
- Hour 11: Working with text: split/join
- Hour 12: Debugging habits (Basics level)

Hour 9: Comparisons + Boolean Logic

Learning Outcomes

- Use comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`)
- Explain the difference between `==` (comparison) and `=` (assignment)
- Combine conditions with `and`, `or`, and `not`
- Chain comparison operators in Python
- Identify boundary condition mistakes (off-by-one errors)

Comparison Operators

The Six Comparisons

```
age = 18

age == 18   # True   (equal to)
age != 21   # True   (not equal to)
age < 21    # True   (less than)
age > 16    # True   (greater than)
age <= 18   # True   (less than or equal)
age >= 18   # True   (greater than or equal)
```

Critical Rule

`=` **assigns** a value — `==` **compares** values

```
x = 5       # assignment
x == 5      # comparison → True
```

Boolean Logic: and / or / not

Combining Conditions

```
age = 20
has_id = True

# Both must be True
age >= 18 and has_id        # True

# At least one must be True
age >= 21 or has_id         # True

# Flip the result
not has_id                  # False
```

Chaining Comparisons

```
score = 85
# Instead of: score >= 70 and score <= 100
```

Truthy and Falsy (Preview)

Quick Peek

- **Falsy values**: `0`, `0.0`, `""`, `None`, `False`
- Everything else is **truthy**

```
name = ""
if name:
    print("Has a name")
else:
    print("Name is empty")   # ← this prints
```

> 💡 We'll see more of this as we work with conditionals.

Demo: Age Gating

Watch for…

- Two comparisons combined with `and`
- Clean boundary handling

```
age = int(input("Enter your age: "))

if age >= 13 and age < 18:
    print("Teen access granted")
elif age >= 18:
    print("Adult access granted")
else:
    print("Access denied — too young")
```

Expected Output (age = 15)

```
Enter your age: 15
```

Lab: Eligibility Checker

Time: 25–35 minutes

Tasks

- Ask the user for their age (use `input()` + `int()`)
- Determine eligibility for 3 brackets:
  - **Child**: age < 13
  - **Teen**: age 13–17 (inclusive)
  - **Adult**: age 18+
- Print a clear message for each bracket

Completion Criteria

✓ Correct branching for boundary values (12, 13, 17, 18)
✓ Uses `>=` and `<` appropriately

Common Pitfalls (Hour 9)

⚠️ **Off-by-one boundaries — test edge values (12, 13, 17, 18)**
⚠️ **Comparing strings to numbers — `input()` returns a string!**
⚠️ **Using `=` instead of `==` — assignment vs comparison**
⚠️ **Forgetting `int()` conversion — `"18" >= 13` is a TypeError**

Quick Check (Hour 9)

Question: What's the difference between `==` and `=`?

Expected Answer:

- `=` assigns a value to a variable

Hour 10: String Formatting with F-Strings

Learning Outcomes

- Use f-strings for readable output (`f"{var}"`)
- Format numbers to fixed decimals (`{value:.2f}`)
- Apply width and alignment specifiers
- Explain why f-strings are better than concatenation

F-String Basics

Simple Variable Insertion

```python
name = "Alice"
age = 25

print(f"Hello, {name}!")
print(f"{name} is {age} years old")
```

Expressions Inside Braces

```python
price = 49.99
tax_rate = 0.08

print(f"Total: {price * (1 + tax_rate)}")
```

> 💡 Anything valid Python can go inside `{}`

Formatting Numbers

Fixed Decimal Places

```
price = 19.999
tip = 3.5

print(f"Price: ${price:.2f}")      # Price: $20.00
print(f"Tip:   ${tip:.2f}")        # Tip:   $3.50
```

Width and Alignment

```
# Right-align in a 10-character field
print(f"{'Item':<10} {'Price':>8}")
print(f"{'Coffee':<10} {4.50:>8.2f}")
print(f"{'Sandwich':<10} {8.75:>8.2f}")
```

```
Item        Price
Coffee         4.50
Sandwich       8.75
```

Why F-Strings Beat Concatenation

Concatenation (Harder to Read)

```
# ❌ String concatenation
print("Hello, " + name + "! You are " + str(age) + " years old.")
```

F-String (Cleaner)

```
# ✓ F-string
print(f"Hello, {name}! You are {age} years old.")
```

Benefits

- No manual `str()` conversion needed
- Reads left-to-right naturally
- Format specifiers built in

Demo: Upgrade Tip Calculator Output

Watch for…

- Reformatting existing output with f-strings
- Consistent decimal formatting
- Alignment for clean columns

```
bill = 85.50
tip_percent = 18
tip = bill * (tip_percent / 100)
total = bill + tip

tip_label = f"Tip ({tip_percent}%):"

print(f"{'Bill:':<12} ${bill:>8.2f}")
print(f"{tip_label:<12} ${tip:>8.2f}")
print(f"{'Total:':<12} ${total:>8.2f}")
```

Lab: Upgrade Tip Calculator

Time: 25–35 minutes

Tasks

- Rewrite your Tip Calculator output using f-strings
- Show bill, tip, and total each on a separate labeled line
- Format all money values to 2 decimal places
- Align numbers in a clean column

Completion Criteria

✓ Clean, consistent output formatting
✓ All monetary values show exactly 2 decimal places
✓ No type errors
✓ Uses f-strings (not concatenation or `str.format()`)

Common Pitfalls (Hour 10)

⚠️ **Forgetting the leading** `f` — `"{name}"` prints literally
⚠️ **Using braces incorrectly** — `f"{{name}}"` prints `{name}`
⚠️ **Mixing quotes** — use different quotes inside vs outside
⚠️ **Wrong format spec** — `.2f` not `.2` for fixed decimals

```
# ❌ Missing the f
print("{name} is {age}")     # literal text

# ✓ With the f
print(f"{name} is {age}")    # variable values
```

Quick Check (Hour 10)

Question: Why is f-string formatting usually better than `+` concatenation?

Expected Answer:

- No need for `str()` conversion
- Easier to read (variables inline)
- Built-in format specifiers for numbers

Hour 11: Working with Text — split() and join()

Learning Outcomes

- Split sentences into lists of words with `.split()`
- Join words back into a string with `' '.join()`
- Count words in a sentence
- Find the longest word in a text string

The split() Method

Default: Split on Whitespace

```
sentence = "Hello Python World"
words = sentence.split()
print(words)        # ['Hello', 'Python', 'World']
print(len(words))   # 3
```

Split on a Custom Delimiter

```
data = "Alice,Bob,Charlie"
names = data.split(",")
print(names)        # ['Alice', 'Bob', 'Charlie']
```

Accessing Individual Words

```
words = "Hello Python World".split()
print(words[0])     # 'Hello'
```

The join() Method

Combining Words into a String

```
words = ['Hello', 'Python', 'World']

# Join with a space
result = ' '.join(words)
print(result)       # 'Hello Python World'

# Join with hyphens
result = '-'.join(words)
print(result)       # 'Hello-Python-World'

# Join with ' | '
result = ' | '.join(words)
print(result)       # 'Hello | Python | World'
```

Key Pattern

The separator goes **before** `.join()`:

Counting Words and Finding the Longest

Word Count

```
sentence = "The quick brown fox jumps"
words = sentence.split()
print(f"Word count: {len(words)}")   # 5
```

Longest Word

```
sentence = "The quick brown fox jumps"
words = sentence.split()

longest = ""
for word in words:
    if len(word) > len(longest):
        longest = word

print(f"Longest word: {longest}")    # 'quick' or 'brown' or 'jumps'
```

Demo: Word Count and Rebuild

Watch for…

- `.split()` producing a list
- `len()` counting words
- `.join()` rebuilding with a new separator

```
sentence = input("Enter a sentence: ")
words = sentence.split()

print(f"Word count: {len(words)}")
print(f"Hyphenated: {'-'.join(words)}")
```

Expected Output

```
Enter a sentence: Python is great
Word count: 3
Hyphenated: Python-is-great
```

Lab: Sentence Stats

Time: 25–35 minutes

Tasks

- Ask the user to input a sentence
- Print the word count
- Print the longest word
- Print a version where words are joined by `|`

Example Output

```
Enter a sentence: The quick brown fox jumps
Word count: 5
Longest word: quick
Pipe version: The | quick | brown | fox | jumps
```

Common Pitfalls (Hour 11)

⚠️ **Punctuation attached to words** — `"hello,"` counts as one word

⚠️ **Empty input edge case** — `"".split()` returns `[]`

⚠️ **Forgetting that split returns a list** — not a string

⚠️ **join() requires strings** — `', '.join([1, 2, 3])` fails

```
# ⚠️ This fails
numbers = [1, 2, 3]
result = ', '.join(numbers)  # TypeError!

# ✓ Convert to strings first
result = ', '.join(str(n) for n in numbers)
```

Quick Check (Hour 11)

Question: What type does `.split()` return?

Expected Answer: A **list** of strings

Hour 12: Debugging Habits (Basics Level)

Learning Outcomes

- Read and interpret common error messages
- Identify NameError, TypeError, ValueError, IndexError
- Use print-debugging strategically to isolate problems
- Apply a systematic debugging process

Reading Error Messages (Tracebacks)

Anatomy of a Traceback

```
Traceback (most recent call last):
  File "calculator.py", line 5, in <module>
    total = bill + tip_amount
NameError: name 'tip_amount' is not defined
```

What to Look For (Bottom to Top)

1. **Error type**: `NameError`
2. **Error message**: `name 'tip_amount' is not defined`
3. **File and line**: `calculator.py`, line 5
4. **Code context**: `total = bill + tip_amount`

> 💡 Read tracebacks from the **bottom up** — the error type is the most important clue

The Four Common Error Types

NameError — Undefined Variable

```
print(mesage)    # ❌ Typo: 'mesage' instead of 'message'
```

TypeError — Wrong Type for Operation

```
age = input("Age: ")
print(age + 5)   # ❌ Can't add str + int
```

ValueError — Right Type, Wrong Value

```
number = int("hello")   # ❌ Can't convert "hello" to int
```

IndexError — Out of Range

```
words = ["hello", "world"]
```

Print-Debugging Strategy

The Process

1. **Reproduce** the error (run the script)
2. **Read** the traceback (bottom up)
3. **Hypothesize** what's wrong
4. **Add print statements** to confirm
5. **Fix** one thing at a time
6. **Re-run** after each fix

```
# Step 4: Add diagnostic prints
print(f"DEBUG: bill = {bill}, type = {type(bill)}")
print(f"DEBUG: tip = {tip}, type = {type(tip)}")
total = bill + tip   # ← error is here
```

Key Habit

Demo: Break and Fix

Watch for…

- Purposely broken script
- Reading the traceback
- Adding print statements to isolate the failure

```
# Broken script — 3 bugs hidden
name = input("Enter name: ")
age = input("Enter age: ")
birth_year = 2025 - age            # Bug 1: age is a string
greeting = f"Hello, {nmae}!"       # Bug 2: typo in variable name
scores = [85, 92, 78]
print(f"Top score: {scores[3]}")  # Bug 3: index out of range
```

Fix Each in Order

1. `age = int(input("Enter age: "))`

Lab: Debugging Drill

Time: 25–35 minutes

Tasks

- You'll receive **3 broken scripts**
- For each script:
    i. Run it and read the error message
    ii. Hypothesize what's wrong
    iii. Add print statements to confirm
    iv. Fix the bug
    v. Write a 1-sentence explanation of what went wrong

Broken Script 1 — NameError

Debugging Drill — Completion Criteria

✓ All 3 scripts fixed and running without errors

✓ Learner can explain each fix in one sentence:

- Script 1: `username` should be `user_name` (variable name typo)
- Script 2: `price` needs `float()` conversion (string × float fails)
- Script 3: Use `colors[2]` or `colors[-1]` (index 3 doesn't exist)

Common Pitfalls (Hour 12)

⚠️ **Changing multiple things at once — fix one bug, re-run**
⚠️ **Not re-running after each fix — always verify**
⚠️ **Ignoring the error message — read it first!**
⚠️ **Leaving debug prints in final code — clean up when done**

Optional: Python 2 vs Python 3 (Quick Note)

Key Differences (When You See Old Code Online)

- **Print**: Python 2 `print "hello"` → Python 3 `print("hello")`
- **Division**: Python 2 `5 / 2 → 2` → Python 3 `5 / 2 → 2.5`
- **Input**: Python 2 `raw_input()` → Python 3 `input()`
- **Strings**: Python 2 ASCII default → Python 3 Unicode default

Quick Check (Hour 12)

Question: You see this error: `TypeError: can only concatenate str (not "int") to str`. What's likely wrong and how do you fix it?

Expected Answer: You're trying to add a string and an integer — use `str()` to convert the int, or use an f-string.

Session 3 Wrap-Up

What We Covered Today

Hour 9: Comparisons + Boolean Logic

- Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Combining conditions with `and`, `or`, `not`
- Eligibility Checker lab

Hour 10: String Formatting with F-Strings

- `f"{variable}"` syntax
- Number formatting: `{value:.2f}`
- Tip Calculator upgrade lab

Hour 11: Text Processing: split/join

Scope Guardrail Reminder

Stay in Basics Scope

✓ Comparison operators and boolean logic
✓ F-string formatting (basics)
✓ split() and join() for text processing
✓ Print-debugging and reading tracebacks

Not Yet (Advanced Topics)

✗ Regular expressions
✗ try/except error handling
✗ List comprehensions
✗ Advanced string formatting (str.format(), %-style)
✗ Custom exception classes

Homework / Practice

Recommended Exercises

1. Build a grade calculator that uses comparisons to assign letter grades
2. Create a formatted receipt with f-string alignment
3. Write a word frequency counter (split a sentence, count occurrences)
4. Debug a multi-bug script (combine all error types from today)

Next Session Preview

Session 4 (Hours 13–16)

- Hour 13: Lists fundamentals — create, index, slice, mutate
- Hour 14: Iterating lists with `for` loops — accumulators & averages
- Hour 15: Nested lists for table-like data — rows & columns
- Hour 16: Checkpoint 2 — Lists + string handling

Questions?

Remember:

- `==` compares, `=` assigns
- Always use f-strings for formatted output
- `.split()` returns a list, `.join()` builds a string
- Debug one fix at a time — always re-run after each change

Thank You!

See you in Session 4!